

```

<!--
{
  "availability" : [
    "iOS: 8.0.0 -",
    "iPadOS: 8.0.0 -",
    "macCatalyst: 13.0.0 -",
    "macOS: 10.10.0 -",
    "tvOS: 9.0.0 -",
    "visionOS: 1.0.0 -",
    "watchOS: 2.0.0 -"
  ],
  "documentType" : "symbol",
  "framework" : "Swift",
  "identifier" : "/documentation/Swift/CVarArg",
  "metadataVersion" : "0.1.0",
  "role" : "Protocol",
  "symbol" : {
    "kind" : "Protocol",
    "modules" : [
      "Swift"
    ]
  },
  "preciseIdentifier" : "s:s7CVarArgP"
},
{
  "title" : "CVarArg"
}
-->

```

CVarArg

A type whose instances can be encoded, and appropriately passed, as elements of a C ``va_list``.

```

...

```

```

protocol CVarArg
{
}

```

Overview

You use this protocol to present a native Swift interface to a C “varargs” API. For example, a program can import a C API like the one defined here:

```

...c
int c_api(int, va_list arguments)

```

To create a wrapper for the ``c_api`` function, write a function that takes ``CVarArg`` arguments, and then call the imported C function using the ``withVaList(_:_:)`` function:

```

...
func swiftAPI(_ x: Int, arguments: CVarArg...) -> Int {
    return withVaList(arguments) { c_api(x, $0) }
}

```

Swift only imports C variadic functions that use a ``va_list`` for their arguments. C functions that use the ``...`` syntax for variadic arguments are not imported, and therefore can’t be called using ``CVarArg`` arguments.

If you need to pass an optional pointer as a ``CVarArg`` argument, use the ``Int(bitPattern:)`` initializer to interpret the optional pointer as an ``Int`` value, which has the same C variadic calling conventions as a pointer

on all supported platforms.

> Note: Declaring conformance to the `CVarArg` protocol for types defined
> outside the standard library is not supported.

Relationships

Conforming Types

[`Bool`](/documentation/Swift/Bool)

[`Dictionary`](/documentation/Swift/Dictionary)

[`UnsafeMutablePointer`](/documentation/Swift/UnsafeMutablePointer)

[`UInt32`](/documentation/Swift/UInt32)

[`Int32`](/documentation/Swift/Int32)

[`Set`](/documentation/Swift/Set)

[`AutoreleasingUnsafeMutablePointer`](/documentation/Swift/AutoreleasingUnsafeMutablePointer)

[`Float`](/documentation/Swift/Float)

[`Int16`](/documentation/Swift/Int16)

[`OpaquePointer`](/documentation/Swift/OpaquePointer)

[`Int8`](/documentation/Swift/Int8)

[`UInt64`](/documentation/Swift/UInt64)

[`UInt16`](/documentation/Swift/UInt16)

[`UnsafePointer`](/documentation/Swift/UnsafePointer)

[`String`](/documentation/Swift/String)

[`UInt8`](/documentation/Swift/UInt8)

[`Int64`](/documentation/Swift/Int64)

[`Float80`](/documentation/Swift/Float80)

[`UInt`](/documentation/Swift/UInt)

[`Int`](/documentation/Swift/Int)

[`Double`](/documentation/Swift/Double)

[`Array`](/documentation/Swift/Array)

Copyright © 2026 Apple Inc. All rights reserved. | [Terms of Use](https://www.apple.com/legal/internet-services/terms/site.html) | [Privacy Policy](https://www.apple.com/privacy/privacy-policy)